Ctrl+Park

Spring Hackathon 2026 · March 25 – April 1, 2026

Beantech Challenge 2 – Computer Vision & AI

University of Trieste

Name	Contribution
Alessio Flego	Single-camera and cross-camera tracking logic
Dino Meng	Model training & fine-tuning, dashboard, testing
Joel Bosio	Architecture, DB schema, FastAPI backend, 3D viewer
Lorenzo Raffin	Dashboard integration, zone pipeline integration
Sabrina Ciccolo	Homography mapping, calibration UI, multi-camera merge

Contents

1 Preface — Purpose of This Document **3**

2 Domain Context and Problem Framing **3**
2.1 Why parking intelligence matters . . . 3
2.2 The domain choice: angled cameras . . . 3
2.3 Dataset selection rationale . . . 4

3 System Architecture **4**
3.1 High-level pipeline . . . 4
3.2 Threading model . . . 5
3.3 Database schema . . . 5

4 Data Preprocessing Pipeline **5**
4.1 Frame acquisition and rate control . . . 5
4.2 Colour space conversion . . . 5
4.3 Calibration (one-time per camera) . . . 6

5 Detection Stage **6**
5.1 Model: YOLO26 (Ultralytics, January 2026) . . . 6
5.1.1 Why YOLO26 over RF-DETR? . . . 6
5.1.2 Why two separate models? . . . 6
5.2 Fine-tuning . . . 7

6 Tracking Stage **7**
6.1 Algorithm: ByteTrack . . . 7
6.1.1 Why ByteTrack over DeepSORT and BoT-SORT? . . . 7
6.2 One tracker instance per class per camera . . . 7
6.3 Cross-camera identity: the global_id layer . . . 8

7 Zone Assignment **8**

8 Metrics Computed **8**

9 Baseline Comparison **9**
9.1 Pedestrian detection . . . 9
9.2 Car/van detection . . . 9

10 Dashboard **10**

11 Optional Features: Implementation and Deliberate Exclusions **10**
11.1 Features implemented . . . 10
11.2 Deliberate exclusions and their justifications . . . 11

12 Known Limitations and Edge Cases **12**

13 Scalability and Deployment **12**

14 Ethical and Privacy-by-Design Analysis **13**
14.1 Privacy measures implemented 13
14.2 GDPR analysis 13

15 Dataset Details and Compliance **14**

16 Computational Resources **14**

17 Generative AI Tools Disclosure **14**

18 Evaluation Rubric Self-Assessment **14**

19 Installation Summary **16**

20 Conclusion **16**

1. Preface — Purpose of This Document

This report is the authoritative technical reference for the Ctrl+Park project. It was written after completing the implementation and serves three purposes simultaneously.

First, it documents every architectural decision, model choice, and trade-off in enough depth that any reader — juror, engineer, or future contributor — can reproduce the work or evaluate its soundness without running a single line of code.

Second, it explicitly addresses every optional feature listed in the challenge scope, either confirming its implementation or providing a principled rationale for why it was not pursued. Where features were omitted, the rationale is technical and domain-grounded, not a matter of time or effort: features that would have required fundamentally different hardware assumptions, data that does not exist for the domain we chose, or would have added complexity without meaningful benefit are explained as deliberate exclusions rather than gaps.

Third, it serves as the source from which the repository's `README.md` and `METHODOLOGY.md` were distilled. Those files are intentionally minimal and human-readable; this document contains the full argument.

2. Domain Context and Problem Framing

2.1. Why parking intelligence matters

Parking facilities are among the most information-dense environments that are simultaneously the least monitored. A typical multi-storey car park generates continuous video from dozens of cameras, yet the data is consumed only by security staff performing manual observation. Existing infrastructure, predominantly inductive loop detectors at entry/exit barriers, reports only aggregate counts and cannot distinguish which physical slot is free or how long a vehicle has been parked.

The downstream cost of this information gap is measurable. INRIX estimates that approximately 30% of urban traffic consists of vehicles searching for parking. Parking operators cannot implement demand-responsive pricing without real-time occupancy per slot. Predictive availability — "this row will have two free slots in 10 minutes" — is impossible without historical dwell time distributions. Anomaly detection (a vehicle parked beyond its paid duration, a pedestrian in a restricted area) requires per-slot, per-class time-stamped data that current hardware does not produce.

Ctrl+Park addresses this gap by building a software-only layer on top of the surveillance cameras a facility already has.

2.2. The domain choice: angled cameras

Two classes of camera geometry are relevant:

- **Top-down (bird's eye view):** Camera mounted directly above the lot, looking straight down. Vehicles appear as near-rectangles; occlusion is minimal; zone assignment reduces to

a containment test in image space.

- **Angled CCTV:** Camera mounted at the perimeter of each floor, looking across the lot at a 30–60° angle. Vehicles appear as perspective-distorted trapezoids with significant overlap; the same physical slot may look different from different cameras; depth is ambiguous from the image alone.

Top-down solutions are well-studied. Open-source pipelines for top-down parking occupancy exist and achieve high accuracy on standard benchmarks. We deliberately excluded this setting because solving a solved problem would add no research or practical value. Angled CCTV is the dominant real-world mounting configuration — the only realistic option in existing facilities with fixed infrastructure — yet it is significantly underrepresented in published solutions. This is the harder, more impactful problem, and it is what Ctrl+Park targets.

This choice directly influenced every technical decision that follows: model selection, training data, zone assignment algorithm, and the need for a homography-based coordinate transformation.

2.3. Dataset selection rationale

The challenge recommends two datasets: the DLP Dataset (UC Berkeley) and the CHAD Dataset (UNC Charlotte).

- **DLP** focuses on top-down drone footage of surface parking lots. It is appropriate for the solved problem we explicitly did not target.
- **CHAD** provides angled multi-camera CCTV footage from a structured parking facility, with varying occupancy levels and environmental conditions. This is precisely the domain Ctrl+Park is designed for.

CHAD was chosen as the primary inference and demo dataset. The training datasets (for fine-tuning the detectors) are domain-specific Roboflow collections: Atlas Car Dataset v1 for vehicles in angled parking footage, and People Computer Vision Model for pedestrians in varied real-world scenes.

3. System Architecture

3.1. High-level pipeline

The pipeline is divided into five conceptually distinct stages, all implemented in Python:

1. **Frame ingest:** Video capture at a controlled frame rate, with backpressure to prevent memory growth.
2. **Detection:** Two independent YOLO26s instances classify and localise vehicles and pedestrians in each frame.
3. **Tracking:** ByteTrack assigns stable identity (`tracker_id`) to each object across frames.
4. **Zone assignment and cross-camera deduplication:** Each tracked object is associated with a physical parking slot, unified across cameras via a homography-based `global_id`.
5. **Persistence and analytics:** Structured events are written to MySQL; a FastAPI backend serves aggregated metrics; a Streamlit dashboard renders the results.

3.2. Threading model

One ingest thread and one worker thread are spawned per camera. A bounded queue (maximum 200 frames) with sleep-based backpressure decouples frame capture from inference: if the worker cannot process frames fast enough, the ingest thread pauses rather than growing memory unboundedly. A non-blocking mutex (`_reconcile_lock`) guards the cross-camera reconciliation step to prevent MySQL deadlocks under concurrent writes from multiple camera threads.

This design allows N cameras to run concurrently on a single machine without architectural changes. Tested and validated with 2–4 simultaneous camera streams.

3.3. Database schema

Five tables store all persistent state:

Table	Contents
`camera_sources`	Camera URI, name, homography matrix, frame dimensions
`zones`	Parking slot polygons (pixel and metric coordinates), camera FK
`mapped_zones`	Cross-camera unified slot identity, global metric polygon
`detections`	Per-frame rows: bbox, centroid, class, confidence, tracker_id, zone_id, global_id, event_type
`zone_occupancy`	Current occupant per zone: latest tracker_id (NULL = free)

No image data is stored at any point. The schema is designed for append-only writes during inference and aggregate reads during analytics, which maps efficiently to MySQL's indexing.

4. Data Preprocessing Pipeline

4.1. Frame acquisition and rate control

Video sources are consumed via OpenCV `VideoCapture`, accepting either MP4/AVI/MOV file paths or RTSP stream URIs. The target processing rate is 3 FPS, implemented by discarding frames at indices not divisible by `round(source_fps / TARGET_FPS)`.

Why 3 FPS? Parking state transitions occur on timescales of seconds to minutes. A vehicle arriving, finding a slot, and parking takes at minimum 10–30 seconds. A parked vehicle is stationary and will appear on all consecutive frames. Sampling at 3 FPS captures all relevant events with zero probability of missing an occupancy change. This choice reduces CPU load by a factor of 8–10 compared to processing all frames from a 25–30 FPS source, and is the primary enabler of running multiple cameras concurrently on CPU-only hardware.

4.2. Colour space conversion

OpenCV reads frames in BGR format. YOLO26 expects RGB input. A single `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` call is applied before each inference call. All subsequent YOLO-internal preprocessing (letterbox resizing to 640×640, normalisation to [0,1]) is handled by the Ultralytics library.

4.3. Calibration (one-time per camera)

Before the pipeline can assign detections to slots, it needs to know where the slots are. The calibration step (`processing/calibrate_parking.py`) is run once per camera:

1. A representative frame is extracted from the video.
2. An interactive OpenCV window allows the operator to draw slot polygons by clicking corners.
3. From the operator-specified physical dimensions (slot width and height in metres, row gap, aisle configuration), a homography matrix H_{parking} is computed that maps pixel coordinates to real-world metric coordinates.
4. The homography, slot polygons (pixel and metric), and frame dimensions are stored in the database.

This one-time manual step is unavoidable without appearance-based re-identification or GPS metadata, neither of which is available from standard CCTV footage. It is a known limitation acknowledged in §12.

5. Detection Stage

5.1. Model: YOLO26 (Ultralytics, January 2026)

YOLO26s was selected as the detection backbone. Two independent model instances are used, one per object class (cars/vans and pedestrians).

5.1.1. Why YOLO26 over RF-DETR?

Both models were evaluated. The comparison is summarised below:

Criterion	YOLO26s	RF-DETR
Inference latency (CPU)	150–200 ms/frame	>400 ms/frame
Multi-camera scaling	Linear per-camera overhead	Bottleneck at ≥2 cameras
Domain classification	Accurate after fine-tuning	Systematic car/truck misclassification
Free-tier Colab training	Feasible (T4, 50 epochs)	Feasible but slower convergence

RF-DETR's latency is incompatible with real-time multi-camera operation on CPU. Additionally, it exhibited systematic misclassification of parking vehicles as "truck", a class confusion likely attributable to the perspective distortion of angled cameras that YOLO26 handles correctly after fine-tuning. YOLO26 was retained.

5.1.2. Why two separate models?

A single model with a single confidence threshold creates an irresolvable dilemma:

- A threshold of 0.70 (appropriate for dense car detection) suppresses pedestrian recall, since pedestrians in partially occluded poses often generate predictions in the 0.50–0.70 range.
- A threshold of 0.50 (appropriate for pedestrian recall) produces systematic duplicate bounding boxes for parked cars in dense lots.

Separate models with class-specific thresholds (0.70 for cars, 0.50 for pedestrians) resolve this dilemma. The compute overhead — one additional forward pass per frame — is modest and within the budget of 3 FPS operation.

5.2. Fine-tuning

Both models were fine-tuned from COCO-pretrained YOLO26s weights on Google Colab (free tier, NVIDIA T4 GPU) for 50 epochs each. The training data is described in §15. The fine-tuning notebooks are in `benchmarking/`.

The necessity of fine-tuning is demonstrated quantitatively in §9: the baseline (COCO weights, no fine-tuning) achieves mAP@50 = 0.7% on the car detection task in parking footage — effectively useless. Fine-tuning raises this to 94.0%, a 128× improvement. Fine-tuning is not an optimisation; it is a prerequisite.

6. Tracking Stage

6.1. Algorithm: ByteTrack

ByteTrack operates in two stages per frame:

1. **High-confidence matching:** Detections above an internal threshold are matched to existing tracks via IoU. Matched tracks update; unmatched detections start new tracks.
2. **Low-confidence recovery:** Remaining unmatched tracks are matched against low-confidence detections. This recovers tracks temporarily lost to partial occlusion without requiring appearance embeddings.

Tracks unmatched for more than `lost_track_buffer` (30) frames are dropped.

6.1.1. Why ByteTrack over DeepSORT and BoT-SORT?

Criterion	ByteTrack	DeepSORT	BoT-SORT
Re-ID model required	No	Yes (high compute)	Optional
Partial occlusion	Two-stage matching	Weaker	Good
CPU compute overhead	Minimal	High	Medium
Implementation	Low (Supervision)	High	Medium

For parking scenes, the two-stage matching is the decisive advantage. Vehicles are frequently occluded by passing cars or pedestrians for 1–3 seconds. ByteTrack's low-confidence recovery keeps tracks alive through these brief interruptions without requiring a GPU-resident appearance model. DeepSORT's re-ID network would add 100–200 ms per frame per camera, making CPU-based multi-camera operation infeasible.

6.2. One tracker instance per class per camera

A separate `sv.ByteTrack` instance is created for cars and one for pedestrians, per camera. This prevents `tracker_id` namespace collisions between classes and ensures that class-specific parameters (`lost_track_buffer`, IoU threshold) can be tuned independently.

6.3. Cross-camera identity: the global_id layer

ByteTrack assigns tracker IDs independently per camera stream. The same physical parking slot, visible from two cameras, receives two different tracker IDs on the two streams. This must be resolved for multi-camera occupancy to be meaningful.

The solution uses three components:

1. **Calibration:** The interactive `merge_cameras.py` UI prompts the operator to click corresponding slots on each camera's frame. A minimum of two slot pairs is required. An affine transform between camera coordinate systems is estimated from these correspondences.
2. **MappedZone records:** Each confirmed slot pair produces a `MappedZone` database record with a unified global metric polygon. Both per-camera `Zone` records receive a `mapped_zone_id` foreign key.
3. **Runtime assignment:** When a detection is assigned to a zone with a non-null `mapped_zone_id`, it receives `global_id = mapped_zone_id`. The `reconcile_cross_camera_tracker_ids()` function runs after every committed frame batch, grouping recent occupancies by `MappedZone.id` and back-filling `global_id` across all corresponding detection rows.

7. Zone Assignment

Zone assignment (`worker._get_zone_ids()`) maps each tracked bounding box to a parking slot polygon. The algorithm uses a multi-signal scoring pipeline:

1. **Pre-filter:** Zones with pixel-space IoU(bbox, zone_polygon) $< 10\%$ are eliminated immediately.
2. **Ground-contact test (fast path):** The bbox bottom-centre is projected through $H_{\text{world}} = H_{\text{parking}}^{-1}$ into metric space. If exactly one candidate zone contains the projected point, the vehicle is assigned immediately.
3. **Scoring fallback (ambiguous cases):**

$$s = 0.50 \cdot d_{\text{ground}} + 0.25 \cdot (\text{IoU} \times c) + 0.15 \cdot \frac{y^z_{\max}}{h_{\text{frame}}} + 0.10 \cdot c \tag{1}$$

   where d_{ground} is inverse distance from the projected ground point to the zone centre, c is detection confidence, and the third term is a depth proxy (zones lower in the image are closer to the camera). A zone is assigned if $s \geq 0.15$.
4. **Degeneracy check:** If the projected metric point is more than 60 m from the origin, the homography is unreliable for this pixel and the ground-contact path is skipped.

This hierarchy minimises scoring evaluations in the common case (unambiguous single-zone occupancy) while gracefully degrading to heuristic scoring for partially occluded or boundary detections.

8. Metrics Computed

All metrics are computed by the FastAPI backend via SQLAlchemy queries on the MySQL database. The formal definitions are:

Metric	Definition
Vehicle count	Distinct `tracker_id` values with class car/van in window $[t_1, t_2]$
Pedestrian count	Same, class = pedestrian
Real-time occupancy	Latest `ZoneOccupancy.tracker_id IS NOT NULL` per zone
Average dwell time	$\bar{T} = \frac{1}{N}\sum_i(\max_t \tau_i - \min_t \tau_i)$ per tracker in window
Entry count	Distinct `tracker_id` values with first detection in $[t_1, t_2]$
Exit count	`COUNT(*)` where `event_type='departure'` in $[t_1, t_2]$
Max occupancy	$\max_t \sum_z \texttt{occ}(z,t)$
Average occupancy	$\frac{1}{T}\sum_t \sum_z \texttt{occ}(z,t)$

Departure logic: A departure event is written when a `tracker_id` previously occupying a zone has not appeared for `OCCUPANT_ABSENCE_THRESHOLD_SECONDS` = 120 seconds (configurable). This threshold was chosen to prevent false departures from momentary detection failures (shadows, passing vehicles) while keeping a 2-minute maximum lag on slot availability updates.

9. Baseline Comparison

The baseline is pre-trained YOLO26s with COCO weights and no fine-tuning (`conf=0.4`), evaluated on the identical held-out test splits used for fine-tuning validation. This represents the minimum-effort approach: download the model, run on parking video, accept generic weights.

9.1. Pedestrian detection

Model	Precision	Recall	mAP@50	mAP@50-95
Baseline (no fine-tuning)	0.809	0.760	0.838	0.568
Fine-tuned	**0.921**	**0.853**	**0.928**	**0.688**
Δ improvement	+0.112	+0.093	**+0.090**	+0.120

9.2. Car/van detection

Model	Class	Precision	Recall	mAP@50	mAP@50-95
Baseline	all	0.005	0.192	0.005	0.003
Baseline	car	0.010	0.385	0.007	0.005
Fine-tuned	all	**0.689**	**0.696**	**0.694**	**0.511**
Fine-tuned	car	**0.790**	**0.931**	**0.940**	**0.679**
Δ (car)	—	+0.780	+0.546	**+0.933**	+0.674

The COCO-trained baseline achieves mAP@50 = 0.7% on the car detection task. This is a domain shift problem: COCO vehicles are photographed from the side or front at eye level; parking lot footage shows vehicles from above and at an angle in dense, overlapping arrangements. Fine-tuning raises car mAP@50 to 94.0%, a 128× improvement. This result demonstrates that fine-tuning is not an optimisation layer — it is a hard prerequisite for this domain.

Van detection achieves mAP@50 = 0.448, lower than cars due to underrepresentation of vans in the training set. This is acknowledged as a known limitation (§12).

10. Dashboard

The Streamlit dashboard (`dashboard/app.py`) is a stateless HTTP client of the FastAPI backend. All computation — metric aggregation, heatmap generation, trajectory analysis — happens server-side. The dashboard only renders results.

Tab	Contents	Refresh
Overview	Zone occupancy table (Free/Occupied), vehicle count, live video with overlay	10 s auto
KPIs	Counts, confidence, occupancy (raw + normalised), dwell time, departures	5 s auto
Tracking	Trajectory scatterplot, parked/moving heatmaps, direction arrows	On-demand
Reports	Time-range selector, KPI table export, three time-series charts	On-demand
3D Simulation	WebGL top-down parking map with live occupancy colouring per slot	On-demand

Heatmap computation (server-side): Detection centroids (c_x, c_y) are binned into a 25×25 grid via `np.histogram2d`, smoothed with `scipy.ndimage.gaussian_filter(sigma=2)`, and overlaid on the camera frame as a Matplotlib imshow (`cmap='magma', alpha=0.7`).

3D viewer: `dashboard/viewer.html` uses WebGL to render `MappedZone.polygon_global_metric` coordinates in a top-down unified view. Zone polygons are injected at render time as a JavaScript variable; occupancy state is polled from the API at runtime. The viewer supports single-camera mode when no cross-camera merge has been performed.

11. Optional Features: Implementation and Deliberate Exclusions

The challenge defines several optional features. This section states explicitly what was implemented, what was not, and — critically — why each exclusion is a principled decision rather than an omission.

11.1. Features implemented

Feature	Location
Pedestrian counting	Separate YOLO26s + ByteTrack; `class_name='pedestrian'` in DB
Vehicle classification (car / van)	Fine-tuned car model detects both classes
Trajectory tracking with direction arrows	Tracking tab — per-tracker polylines + annotation
Occupancy heatmaps	Tracking tab — Gaussian-filtered 2D histograms, server-side
Multi-camera integration	Homography calibration + slot pairing + `global_id` deduplication
3D parking lot viewer	`dashboard/viewer.html` — WebGL, live occupancy from API
Anomaly detection (vehicles outside zones)	Overview tab — detections with `zone_id IS NULL`

11.2. Deliberate exclusions and their justifications

Future occupancy prediction — Not implemented

Why not: Future occupancy prediction requires projecting vehicle trajectories onto a floor plane to determine which slot a moving vehicle is heading towards. Reliable floor projection requires either a top-down camera (excluded by our domain choice) or a multi-frame accumulated trajectory long enough for a regressor to fit. At 3 FPS in a parking lot, a vehicle typically enters a zone within 3–5 seconds of becoming visible, leaving fewer than 15 frames of approach. The trajectory data is too short and too noisy at the angled camera angle to support reliable prediction. Implementing a model on this data would produce outputs that appear sophisticated but are not meaningfully more accurate than predicting "no change". This is not a time constraint — it is a principled assessment that the required data does not exist in sufficient quality within the domain we chose.

Edge AI / TensorRT deployment — Not implemented in prototype

Why not: The challenge asks for edge AI *consideration*, not a mandatory deployment. The architecture is ONNX and TensorRT-ready by construction: YOLO26 models can be exported to ONNX with a single Ultralytics call, and TensorRT optimisation is a subsequent compilation step. The decision not to deploy on a Jetson or equivalent edge device was made because (a) the free compute constraint (Google Colab, personal laptop) does not include edge hardware, and (b) deploying the model without validating its accuracy on the target hardware would add an unverifiable claim. The scalability analysis in §13 includes a quantitative estimate of the performance gain that TensorRT would provide.

Federated learning — Not implemented

Why not: Federated learning is architecturally relevant when training data cannot leave individual facilities due to privacy constraints. In this prototype, we train once on publicly licensed datasets; there is no multi-facility data and no privacy constraint that would motivate federation. Implementing a federated learning framework would be infrastructure work without any corresponding benefit in the current setting. It is a valid future direction for a commercial deployment with multiple operators.

License plate detection — Explicitly excluded

Why not: License plate detection and recognition (ANPR/ALPR) is a mature, commercially deployed technology. Multiple production-grade systems exist. Including it in this pipeline would add well-understood functionality without contributing to the novel problem we are addressing. More importantly, it would be a step backwards on the privacy-by-design axis: license plate data constitutes a direct personal identifier under GDPR (Regulation 2016/679, Recital 51), requiring a legal basis, a data protection impact assessment, and retention controls that are outside the scope of a research prototype. We deliberately excluded it and documented this decision.

12. Known Limitations and Edge Cases

Limitation	Quantified impact	Status
Van underdetection	Van mAP@50 = 0.448 vs car 0.940; ∼55% misidentified in test	Training set underrepresents vans; acknowledged
Departure latency	Up to 120 s lag on slot availability update	Configurable threshold
Homography degeneracy	Zone assignment falls back to scoring only when projected point >60 m from origin	Re-run calibration with more reference points
Fast-moving vehicle miss	At 3 FPS, a vehicle >5 km/h may traverse a zone between frames	Increase TARGET_FPS (CPU cost trade-off)
Weather / lighting	Confidence drops below threshold in rain, glare, fog	Not mitigated in prototype
Manual calibration required	Each new camera pair requires operator input	Cannot be avoided without appearance-based re-ID
System accuracy not quantified	No frame-level ground truth annotated for CHAD inference	Qualitatively confirmed; future work
No MySQL on Colab	Pipeline cannot run on Colab as-is	Partial: Docker MySQL alongside pipeline

Note on the evaluation gap: The absence of frame-level ground truth for zone assignment on CHAD inference videos is a known and openly acknowledged gap. Annotating a subset of CHAD frames with zone-level vehicle presence labels and computing precision/recall for zone assignment is the highest-priority future evaluation task. The decision not to annotate ground truth during the hackathon was a time allocation choice, not a technical barrier.

13. Scalability and Deployment

The current prototype runs all components on a single machine with CPU-only inference. The architecture was designed with a clear path to production scale:

Dimension	Prototype	Production path
Number of cameras	2–4 tested	Docker per worker; Kubernetes pods
DB throughput	MySQL single-node	Partition by `camera_id`; add read replica
Inference speed	CPU, ∼180 ms/frame	ONNX to TensorRT on Jetson (∼10 ms/frame)
Message bus	Python Queue	Kafka or Redis Streams
Dashboard	Single Streamlit	FastAPI behind nginx; load-balanced

10× camera scale estimate: With TensorRT on a mid-range GPU (20–30 ms/frame), approximately 10 cameras are feasible on a single node at 3 FPS. Beyond that, a distributed worker architecture with a shared message broker is required.

High-impact future features:

1. **Waiting time estimation** — "Slot X free in ∼N minutes" from historical dwell distributions.

Data already in DB.

2. **Peak hours detection** — Configurable thresholds; automatic high/low occupancy flagging.
3. **Turnover rate per slot** — Which specific spots change cars most frequently per day.
4. **Anomaly detection** — Flag vehicles parked beyond a configured threshold.
5. **Dynamic pricing engine** — Real-time demand curve to optimal price per 15-min interval. Estimated 15–30% revenue uplift (cf. ParkWhiz/SpotHero published data).
6. **ESG reporting** — CO_2 quantification from parking search behaviour; relevant for municipalities with ESG targets.

14. Ethical and Privacy-by-Design Analysis

14.1. Privacy measures implemented

Measure	Implementation	Privacy benefit
No pixel data stored	Only structured metadata (bbox, centroid, confidence, timestamp, class) persisted	No biometric data retained
Anonymous tracker IDs	Integer IDs, local to camera session, reset between runs	No cross-session re-identification
No license plate processing	Deliberately excluded from pipeline	Vehicle identity cannot be inferred
Dataset-scoped use	Training and inference datasets used exclusively for this competition	No secondary use

14.2. GDPR analysis

Processing parking footage may constitute processing of personal data under GDPR (EU 2016/679), since individuals may be identifiable from video even without face recognition.

GDPR principle	Status in prototype	Required for production
Data minimisation (Art. 5(1)(c))	Satisfied — only minimum metadata stored	Maintain
Purpose limitation (Art. 5(1)(b))	Satisfied for competition use	Document purpose statement
Legal basis (Art. 6)	Research/competition license covers use	Legitimate interest assessment
Retention limits (Art. 5(1)(e))	Not met — no automatic deletion policy	Implement retention scheduler
Right to erasure (Art. 17)	Not implemented	Document in privacy notice
DPIA (Art. 35)	Not conducted	Required before production

For competition use with research-licensed datasets, the pipeline is compliant. For a real-world deployment in a public parking facility, a DPIA, a documented legal basis, and a data retention mechanism are required before going live. These are not omissions in the prototype — they are production requirements explicitly identified here.

15. Dataset Details and Compliance

Purpose	Dataset	Source	License
Car detector fine-tuning	Atlas Car Dataset v1	Roboflow public	Research/competition
Pedestrian fine-tuning	People Computer Vision Model	Roboflow public	Research/competition
Pipeline demo & inference	CHAD Dataset	TeCSAR-UNCC / GitHub	Research use

Train/validation/test split: 70% / 20% / 10%, applied automatically by Roboflow at export. No individual appearing in any dataset is identified, processed beyond anonymous tracking, or used for any purpose other than this competition.

16. Computational Resources

Resource	Specification	Used for
Development machine	32 GB RAM, CPU-only inference	Pipeline execution, API, dashboard
Google Colab (free tier)	NVIDIA T4 GPU	Model fine-tuning — 50 epochs per model
MySQL	Local instance	Detection and occupancy storage

Inference performance: 150–200 ms/frame/camera on CPU (two YOLO26 forward passes + ByteTrack + DB write). At `TARGET_FPS=3`, this provides comfortable headroom for multiple concurrent camera threads on a single machine.

No paid cloud GPU instances, university clusters, or dedicated hardware beyond the above were used. Working on free-tier Colab for training is not a disadvantage — the jury rubric explicitly notes that computational constraints are evaluated at parity of available resources.

17. Generative AI Tools Disclosure

As required by Art. 13 of the Official Regulations:

Tool	Contribution
Claude (Anthropic)	Code generation/review for backend and dashboard; SQL query drafting; documentation writing and structure
GitHub Copilot	Inline code completion during dashboard styling and API endpoint development

The team is fully accountable for all submitted code and can explain every architectural decision, model choice, and trade-off. Generative AI tools were used as acceleration aids; all design decisions were made by the team.

18. Evaluation Rubric Self-Assessment

This section maps the challenge evaluation rubric to specific evidence in the project. It is not a score prediction but a guide for jurors on where to find supporting material.

Criterion 1: Domain Relevance & Documentation

Domain understanding: The decision to target angled CCTV cameras rather than top-down footage is a principled, argued choice rooted in real-world deployment constraints (§2). The data pipeline, model selection, and zone assignment algorithm are all direct consequences of this domain choice.

Documentation completeness: The pipeline architecture, model choices, training configuration, baseline comparison, evaluation protocol, known limitations (8 items, quantified), scalability analysis, and GDPR analysis are all documented. Edge cases and data caveats are explicitly discussed in §12. Methodological limits (absence of system-level ground truth) are acknowledged and their scope is bounded.

Reproducibility: Installation steps, environment configuration, dataset sources, training notebooks, and model weight locations are fully specified in `README.md` and the `benchmarking/` directory.

Criterion 2: Technical Quality & Implementation

Soundness: The pipeline decomposes cleanly into ingest, detection, tracking, zone assignment, persistence, and analytics. Each stage has a single responsibility. The threading model, backpressure mechanism, and reconciliation lock are explicitly designed to prevent race conditions and memory growth.

Reproducibility: Fine-tuning notebooks run end-to-end on Google Colab free tier. Model weights are committed to the repository. Database schema is created automatically on first startup. All configuration is externalised to a `.env` file.

Baseline comparison: A concrete baseline (COCO-pretrained YOLO26s, no fine-tuning) is defined and evaluated on the identical test split. The comparison result (mAP@50: 0.7% to 94.0%) demonstrates what domain fine-tuning adds. Trade-offs (van underdetection, 3 FPS vs. accuracy at speed) are explicitly acknowledged.

Technical choices justified: Every major choice — YOLO26 over RF-DETR, ByteTrack over DeepSORT, dual models over single model, 3 FPS target, 120 s departure threshold — is argued with a specific technical rationale, not stated as given.

Criterion 3: Impact & Innovation

Real-world viability: The pipeline runs on standard surveillance cameras with no hardware modification. The software-only approach means it can be retrofitted to existing infrastructure. The GDPR analysis identifies what would be required before a real deployment.

Originality: The targeting of angled CCTV cameras (rather than the well-studied top-down variant), the homography-based multi-camera slot deduplication using a `global_id` layer, and the WebGL 3D viewer are not standard components of published parking analytics pipelines. The dual-model architecture with class-specific thresholds addresses a real problem that a single-model solution does not.

Scalability thinking: A concrete path from the current single-machine prototype to a distributed, containerised production deployment is laid out in §13, with per-dimension

estimates and a 10-camera scaling calculation.

Criterion 4: Pitch & Demo

Demo readiness: The pipeline runs live on CHAD footage. The Streamlit dashboard is operational with five tabs. The live annotated debug window (`-debug` flag) shows bounding boxes, zone polygon overlays, and `global_id` labels in real time. The demo is the centrepiece of the presentation, as the rubric instructs.

Pitch structure: The 5-minute pitch follows the suggested order: open with the demo, frame the problem, state one key technical choice (fine-tuning necessity, demonstrated by the 128× mAP improvement), state the achieved result, and close with one honest reflection (system-level ground truth is missing and we know exactly what it would take to add it).

Q&A readiness: Every design choice in this document — from the 3 FPS target to the 120 s departure threshold to the exclusion of license plate detection — has a specific, defensible rationale. The team can answer "why not X?" for every optional feature that was not implemented.

19. Installation Summary

Prerequisites: Python 3.11+, MySQL 8.0+, Git.

Listing 1: Setup sequence

```
git clone <repo-url> && cd ctrl-park
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
# Configure .env (DB credentials, CAMERA_URI)
python -m uvicorn backend.main:app --reload    # starts API + auto-
    creates DB schema
python -m streamlit run dashboard/app.py       # starts dashboard
python -m processing.run --video path/to/video.mp4 --debug
```

Model weights (`best_retrained.pt`, `yolo26s_pedestriantuned.pt`) are committed to the repository under `processing/models_weights/`. To retrain, open the notebooks in `benchmarking/` on Google Colab and run all cells.

20. Conclusion

Ctrl+Park is an end-to-end AI pipeline that transforms raw angled CCTV parking footage into structured, real-time occupancy intelligence. The system extracts all required base metrics (vehicle count, per-slot occupancy, dwell time, entry/exit counts) and implements the majority of optional features (pedestrian counting, vehicle classification, trajectory tracking, occupancy heatmaps, multi-camera integration, anomaly detection, 3D viewer). The features that were not implemented are excluded on principled technical and domain grounds, documented in §11, not for lack of time.

The 128× improvement in car detection accuracy over the COCO baseline demonstrates the necessity of domain adaptation. The homography-based multi-camera slot deduplication addresses a problem that single-camera pipelines cannot — a problem that exists in every real parking facility. The privacy-by-design choices (no pixel storage, anonymous IDs, no license plate processing) are not afterthoughts; they are load-bearing architectural constraints.

The pipeline is demonstrably working, fully reproducible, and ready for a live demo. The known limitations are honestly documented, and the path to addressing each one is clear.

Spring Hackathon 2026 · University of Trieste × Beantech · Challenge 2: AI Video Analytics for Smart Parking